



Análisis de Señales y Sistemas Digitales

Trabajo Práctico N° 2

Sintetizador de Audio

BELSITO, RAMIRO NAHUEL 62641 *rabelsito@itba.edu.ar*

SAMMARTINO, IGNACIO 63053 *isammartino@itba.edu.ar*

MINNITI, MATIAS ADRIAN 64030 *mminniti@itba.edu.ar*

GASTALDI, ROCCO 61659 *rgastaldi@itba.edu.ar*

TERRA, IGNACIO 59558 *terrai@itba.edu.ar*

Profesores: Daniel Andres Jacoby

Fecha de entrega: 28 de Octubre de 2025

Índice

1. Introducción	1
1.1. Síntesis aditiva	2
1.2. ADSR	2
1.3. Síntesis por modulación de frecuencia	3
1.4. Síntesis por Karplus-Strong	4
1.5. Efecto de retardo (Delay)	5
1.6. Efecto de Flanger	6
1.7. Efecto de reverberación/Reverb	6
1.8. Overlap en el Espectrograma	7

1. Introducción

Transformada Rápida de Fourier (FFT)

Para aplicar la *Fast-Fourier Transform* se utiliza el algoritmo de Cooley-Tukey.

Partiendo de:

$$X(k) = \sum_{n=0}^{N-1} x(n)W_N^{kn}, \quad W_N = e^{-j2\pi/N} \quad (1)$$

Siendo $N = 2^r$, se separa la suma en índices pares e impares:

$$X(k) = \sum_{n=0}^{N/2-1} x(2n)W_{N/2}^{kn} + W_N^k \sum_{n=0}^{N/2-1} x(2n+1)W_{N/2}^{kn} \quad (2)$$

Definiendo $x_p(n) = x(2n)$ y $x_i(n) = x(2n+1)$:

$$X(k) = X_p(k) + W_N^k X_i(k) \quad (3)$$

Por la periodicidad de la DFT:

$$X\left(k + \frac{N}{2}\right) = X_p(k) - W_N^k X_i(k) \quad (4)$$

De este modo, se calculan los valores para la mitad del intervalo y se reutilizan para la otra mitad. El algoritmo de Cooley-Tukey repite este proceso recursivamente hasta llegar a transformadas de 1 punto.

Implementación en código

El código realiza la transformada rápida de Fourier mediante un enfoque recursivo que divide el problema original de tamaño N en dos subproblemas de tamaño $N/2$, correspondientes a las muestras pares e impares de la señal de entrada.

```

1 void fft_recursive(const vector<cfloat>& in, vector<cfloat>& out, size_t n, size_t stride) {
2     if (n==1) {
3         out[0] = in[0];
4         return;
5     }
6     size_t m = n/2;
7     vector<cfloat> even(m), odd(m);
8     for(size_t i=0; i<m; i++){
9         even[i] = in[i*2*stride];
10        odd[i] = in[(i*2+1)*stride];
11    }
12    vector<cfloat> Fe(m), Fo(m);
13    fft_recursive(even, Fe, m, 1);
14    fft_recursive(odd, Fo, m, 1);
15    for(size_t k=0; k<m; k++){
16        cfloat t = std::polar(1.0f, float(-2.0f*M_PI*k/n)) * Fo[k];
17        out[k] = Fe[k] + t;
18        out[k+m] = Fe[k] - t;
19    }
20 }
```

1.1 Síntesis aditiva

En este enfoque, el sonido se obtiene como la suma de varios parciales sinusoidales (armónicos) de diferente frecuencia, amplitud y envolvente temporal.

```
vector<float> synthAdditive(int midiNote, float dur, float sr,
    const vector<float>& partialAmps, const vector<ADSR>& partialEnvs)
```

La función recibe los siguientes parámetros:

- **midiNote**: número de nota MIDI que determina la frecuencia fundamental del sonido.
- **dur**: duración total de la nota, en segundos.
- **sr**: frecuencia de muestreo (sample rate) del sistema, en Hz.
- **partialAmps**: vector con las amplitudes relativas de cada parcial armónico.
- **partialEnvs**: vector de objetos **ADSR** (Attack–Decay–Sustain–Release) que definen la envolvente temporal de cada parcial.

Primero, se calcula la frecuencia fundamental del tono mediante la función `midiNoteFreq()`, y se determina el número total de muestras a generar según la duración y la frecuencia de muestreo. Luego, se recorre cada muestra en el dominio temporal y se acumula la contribución de cada parcial armónico. La frecuencia de cada parcial es un múltiplo entero de la fundamental ($f_p = (p + 1)f_0$), y su amplitud instantánea depende tanto del peso asignado en `partialAmps` como de la envolvente temporal correspondiente `partialEnvs[p]`.

El valor instantáneo de la señal se calcula como:

$$s(t) = \sum_{p=0}^{N_p-1} A_p E_p(t) \sin(2\pi(p+1)f_0 t)$$

donde A_p es la amplitud del parcial p , $E_p(t)$ su envolvente, y f_0 la frecuencia fundamental.

Finalmente, la función devuelve un vector con las muestras de la señal sintetizada, representando la nota en formato monoaural. Este método permite construir timbres complejos a partir de la suma controlada de armónicos y sus respectivas envolventes.

1.2 ADSR

La función **ADSR** (Attack–Decay–Sustain–Release) es un modelo matemático utilizado en síntesis de audio para describir la envolvente de amplitud de un sonido. Una envolvente controla cómo evoluciona la amplitud de un sonido a lo largo del tiempo, y el modelo **ADSR** es uno de los más comunes en sintetizadores.

```
1 struct ADSR {
2     float A=0.01f, D=0.05f, S=0.8f, R=0.1f;
3     float sampleRate=44100;
4     float env(float t, float noteLen) const {
5         if(t<0) return 0;
6         if(t < A) return (t/A);
7         else if(t < A+D) return 1.0f - (t-A)/D*(1.0f - S);
8         else if(t < noteLen) return S;
9         else {
10             float rt = t - noteLen;
11             if(rt > R) return 0;
12             return S * (1.0f - rt/R);
```

```

13     }
14   }
15 };

```

Los parametros de la envolvente son tal que:

- $A = 0,01$ s: tiempo de ataque corto, simula un inicio rápido del sonido (p.ej., una tecla de piano).
- $D = 0,05$ s: tiempo de decaimiento breve, la amplitud baja rápidamente después del pico inicial.
- $S = 0,8$: nivel de sostenimiento alto, el sonido se mantiene casi con la misma intensidad.
- $R = 0,1$ s: liberación moderada, el sonido desaparece suavemente al soltar la nota.

Estos valores se asemejan al comportamiento de instrumentos percusivos o de cuerda corta (piano, guitarra).

1.3 Síntesis por modulación de frecuencia

La síntesis por modulación de frecuencia (FM) es una técnica ampliamente utilizada en la generación de sonidos complejos a partir de osciladores sinusoidales simples. En este método, la frecuencia instantánea de una señal denominada *portadora* (*carrier*) se modula en función de otra señal sinusoidal llamada *moduladora* (*modulator*). Esta modulación genera un espectro armónico rico, controlable mediante parámetros que definen la relación entre ambas frecuencias y la profundidad de modulación.

```

vector<float> synthFM(int midiNote, float dur, float sr,
    float carrierRatio, float modRatio, float modIndex,
    const ADSR& env)

```

La función `synthFM` implementa una versión básica de este modelo. A partir del número de nota MIDI se calcula la frecuencia fundamental f_0 , y se definen las frecuencias de la portadora y la moduladora como:

$$f_c = f_0 \cdot \text{carrierRatio}, \quad f_m = f_0 \cdot \text{modRatio}.$$

El índice de modulación I (en el código, `modIndex`) controla la desviación máxima de la frecuencia instantánea de la portadora, y por lo tanto la cantidad de componentes espectrales (bandas laterales) generadas. La señal resultante se expresa como:

$$s(t) = A(t) \sin(2\pi f_c t + I \sin(2\pi f_m t)),$$

donde $A(t)$ es la envolvente temporal (ADSR) aplicada para modelar la dinámica del sonido.

El término interno $I \sin(2\pi f_m t)$ produce una modulación angular periódica que, según la teoría de la FM, da lugar a una serie infinita de componentes espectrales centradas en la frecuencia portadora f_c y espaciadas en múltiplos de la frecuencia moduladora f_m :

$$f_k = f_c \pm k f_m, \quad k = 1, 2, 3, \dots$$

La amplitud relativa de cada componente depende de las funciones de Bessel de primer tipo $J_k(I)$, donde I es el índice de modulación. A medida que I aumenta, se incrementa el número de armónicos significativos y el timbre se vuelve más complejo.

En la implementación, la función recorre todas las muestras temporales ($t = n/sr$) y evalúa directamente la ecuación de la FM. La envolvente `env` controla la amplitud global del sonido a lo largo del tiempo, reproduciendo el comportamiento típico de instrumentos reales (ataque, decaimiento, sostenido y liberación).

```
vector<float> synthFM(int midiNote, float dur, float sr, float carrierRatio,
                    float modRatio, float modIndex, const ADSR& env);
```

1.4 Síntesis por Karplus-Strong

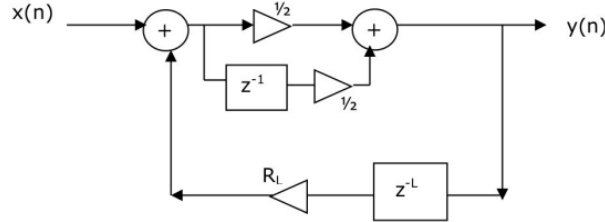


Figura 1.1: Esquema de Karplus-Strong

La función de transferencia (cerrada) es

$$H(z) = \frac{\frac{1}{2}(1 + z^{-1})}{1 - \frac{R_L}{2}(1 + z^{-1})z^{-L}}$$

y los polos son las raíces del denominador:

$$D(z) = 1 - \frac{R_L}{2}(1 + z^{-1})z^{-L} = 0 \iff 2z^{L+1} - R_L(z + 1) = 0.$$

Por tanto la ecuación característica (polinómica) es

$$P(z) = 2z^{L+1} - R_L(z + 1) = 0.$$

Para estudiar la estabilidad marginal sustituimos $z = \alpha e^{j\omega}$:

$$2\alpha e^{j\omega(L+1)} = R_L(\alpha e^{j\omega} + 1).$$

Tomando magnitudes y $\alpha = 1$:

$$2 = R_L |e^{j\omega} + 1| = R_L \cdot 2 |\cos(\frac{\omega}{2})| \Rightarrow R_L |\cos(\frac{\omega}{2})| = 1.$$

De aquí se obtienen las siguientes conclusiones prácticas:

- Si $0 \leq R_L < 1$ entonces no existe ω que cumpla $R_L |\cos(\omega/2)| = 1$, por lo que no hay polos en el contorno $|z| = 1$. Intuitivamente, la ganancia de lazo es menor que 1 y la respuesta se amortigua: *asintóticamente estable*.
- Si $R_L = 1$ pueden existir soluciones con $|z| = 1$ (p. ej. $\omega = 0$ da $|\cos(0/2)| = 1$), lo que produce *estabilidad marginal* (oscilaciones sostenidas / polos en el círculo unidad) dependiendo de L .
- Si $R_L > 1$ existe ω tal que $R_L |\cos(\omega/2)| > 1$ y, por continuidad, habrá raíces con $|z| > 1$: el sistema será *inestable* (crecimiento en el tiempo).

En la práctica se elige $R_L \lesssim 1$ (ligeramente menor que 1) para obtener oscilaciones amortiguadas pero con larga duración (comportamiento típico del algoritmo Karplus-Strong). Para localizar polos concretos y verificar estabilidad se resuelven numéricamente las raíces de $P(z) = 0$ para el L y R_L dados.

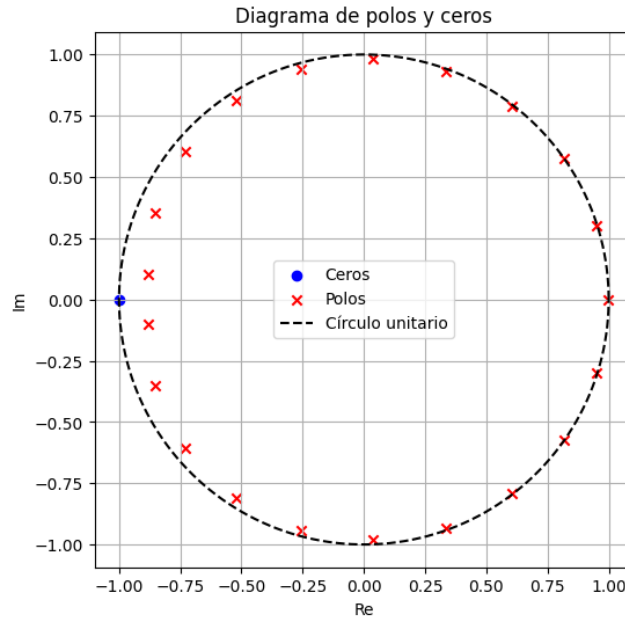


Figura 1.2: Diagrama de Polos y Ceros, tomando $L = 20$ y $RL = 0.99$

1.5 Efecto de retardo (Delay)

El efecto de retardo, o delay, es uno de los procesamiento más utilizados en la producción y síntesis de audio. Su principio se basa en almacenar la señal de entrada y reproducirla nuevamente después de un cierto tiempo, generando una o varias repeticiones (ecos) controladas mediante parámetros de retroalimentación y mezcla.

```
void applyDelay(vector<float>& buf, float sr,
               float delaySec, float fb, float mix)
```

En la función `applyDelay`, el parámetro `delaySec` define el tiempo de retardo en segundos, mientras que `fb` (feedback) controla la cantidad de señal retardada que se reinserta en el búfer de retardo, y `mix` determina la proporción entre la señal directa y la señal procesada en la salida.

El algoritmo utiliza un búfer circular (*ring buffer*) de longitud igual al número de muestras correspondiente al tiempo de retardo:

$$N_d = f_s \cdot t_d,$$

donde f_s es la frecuencia de muestreo y t_d el tiempo de retardo. En cada iteración se realiza el siguiente procedimiento:

- 1.1 Se toma la muestra actual de entrada $x[n]$.
- 1.2 Se recupera la muestra retardada $d[n]$ almacenada en la posición actual del búfer.
- 1.3 Se calcula la salida como una mezcla lineal entre la señal original y la señal retardada:

$$y[n] = (1 - \text{mix}) x[n] + \text{mix} d[n].$$

- 1.4 Se actualiza el búfer de retardo con la suma de la señal original y una fracción de retroalimentación:

$$\text{ring}[i] = x[n] + \text{fb} \cdot d[n].$$

1.5 Se avanza el índice circular en el búfer.

La retroalimentación (**fb**) introduce múltiples repeticiones decrecientes en amplitud, generando el típico efecto de eco progresivo. La mezcla (**mix**) permite controlar la proporción entre la señal seca (*dry*) y la señal procesada (*wet*), mientras que el tiempo de retardo define la separación temporal entre ecos.

1.6 Efecto de Flanger

El flanger es un efecto de modulación basado en la variación periódica del tiempo de retardo aplicado a una señal. Su resultado son interferencias constructivas y destructivas que generan un patrón de cancelaciones y refuerzos armónicos conocido como “peine de filtrado” o *comb filter*, característico del sonido del flanger.

```
void applyFlanger(vector<float>& buf, float sr, float rate = 1.0f,
    float depth = 0.001f, float feedback = 0.5f, float mix = 0.5f)
```

El funcionamiento se basa en un retardo corto y variable en el tiempo, típicamente del orden de milisegundos, modulado por una señal de baja frecuencia (LFO). El parámetro **rate** define la frecuencia del LFO en hercios, mientras que **depth** determina la profundidad máxima de modulación del retardo. El parámetro **feedback** controla la cantidad de señal retardada que se reinserta en el efecto, incrementando la resonancia, y **mix** ajusta la proporción entre señal directa y procesada.

El algoritmo implementa un búfer circular de longitud igual al máximo retardo posible. En cada muestra, se calcula la posición de lectura desplazada según la forma sinusoidal del LFO:

$$\tau(t) = \frac{d_{\max}}{2} [1 + \sin(2\pi f_{\text{LFO}} t)],$$

donde $d_{\max}$ es el retardo máximo y f_{LFO} la frecuencia del oscilador de baja frecuencia. Este retardo fraccionario se interpola linealmente entre las muestras vecinas para obtener un desplazamiento continuo.

A continuación, la muestra retardada se combina con la señal original:

$$y[n] = (1 - \text{mix}) x[n] + \text{mix} x[n - \tau(t)],$$

y se actualiza el contenido del búfer con la suma de la señal original y una fracción de la señal retardada en función del factor de retroalimentación. El movimiento continuo del retardo produce una modulación en fase que se traduce en el característico efecto de barrido del flanger.

La función **applyFlanger** implementa este procedimiento mediante un LFO sinusoidal, interpolación lineal y un búfer circular para almacenar las muestras retardadas, permitiendo reproducir el efecto en tiempo real sobre cualquier señal de audio digital.

1.7 Efecto de reverberación/Reverb

La reverberación es un fenómeno acústico producido por la superposición de múltiples reflexiones del sonido en un recinto. En el dominio digital, este efecto se simula utilizando redes de retardos con realimentación que recrean la persistencia temporal del campo sonoro. El algoritmo clásico de Schroeder constituye una de las implementaciones más simples y efectivas para este propósito, combinando filtros de retardo con retroalimentación (filtros *comb*) y secciones de fase (*all-pass*) para obtener una respuesta densa y difusa.

```
void applyReverb(vector<float>& buf, float sr,
    float roomSize = 1.0f, float damping = 0.5f, float wetLevel = 0.5f)
```


La función `applyReverb` implementa una versión simplificada del reverberador de Schroeder mediante tres retardos de tipo *comb* en paralelo. Los tiempos de retardo, expresados en milisegundos, se ajustan proporcionalmente al parámetro `roomSize`, que representa de forma aproximada el tamaño acústico del recinto. Cada línea de retardo se implementa con un búfer circular que acumula las muestras con una fracción de retroalimentación controlada por el parámetro `damping`, el cual simula la absorción del aire y de las superficies del entorno.

En cada iteración, la muestra de salida se calcula como la suma de la señal directa y de la muestra almacenada en el retardo, multiplicada por un coeficiente de realimentación:

$$y[n] = x[n] + \alpha y[n - L],$$

donde L es la longitud del retardo y α el coeficiente de realimentación dependiente de la amortiguación. La suma de varias líneas con diferentes longitudes de retardo produce una densidad temporal suficiente para emular la respuesta difusa de un espacio real.

Finalmente, la señal reverberada se mezcla con la señal original según el parámetro `wetLevel`, que define la proporción entre señal seca y procesada:

$$z[n] = (1 - w) x[n] + w y[n].$$

La implementación es eficiente en cómputo y permite ajustar la extensión, la atenuación y la mezcla del efecto mediante los parámetros de tamaño de sala, amortiguamiento y nivel de reverberación, generando una sensación espacial convincente a partir de un modelo mínimo.

1.8 Overlap en el Espectrograma

En un espectrograma, la señal se divide en segmentos solapados sobre los cuales se calcula la Transformada de Fourier de Tiempo Corto (STFT). Sea L la longitud de la ventana y H el salto temporal entre ventanas consecutivas (hop size). Se define el *overlap* como la fracción de muestras compartidas entre ventanas:

$$\text{overlap} = 1 - \frac{H}{L} \quad (5)$$

El propósito del solapamiento es evitar la pérdida de información temporal entre ventanas y reducir las discontinuidades en la representación tiempo–frecuencia.

Un requisito fundamental para obtener un espectrograma adecuado es que la suma de las ventanas desplazadas en el tiempo mantenga una energía constante, lo cual se denomina propiedad de *Constant Overlap-Add* (COLA). Esta propiedad garantiza que no existan fluctuaciones artificiales de amplitud entre ventanas.

En el caso particular de la ventana de Hann, cuyo uso es común por su bajo *leakage* espectral, se cumple la condición COLA cuando el salto temporal es:

$$H = \frac{L}{2} \quad (6)$$

lo cual corresponde a un solapamiento del 50 %. En esta configuración, la superposición de ventanas cumple:

$$\sum_{n=-\infty}^{\infty} \text{hann}(t - nH) = 1, \quad \forall t \quad (7)$$

Es decir, cada punto temporal de la señal está cubierto por dos ventanas que se complementan en amplitud, evitando oscilaciones o modulación indeseada de la energía.

En aplicaciones donde se requiere una mayor resolución temporal (mayor suavidad entre ventanas), es común aumentar el solapamiento al 75 % (hop $H = L/4$), a costa de incrementar el costo computacional y la redundancia en los datos.